

Pra-Pemrosesan Data CISA ICS Advisory

Notebook: 01_pra_pemrosesan.ipynb

Tahap: Pra-Pemrosesan Data

Deskripsi: Notebook ini menangani seluruh proses pra-pemrosesan data sebelum masuk ke tahap pemodelan BERTopic. Proses mencakup pemuatan data mentah dari file JSON CISA Advisory, ekstraksi teks relevan, pembersihan karakter spesial, serta normalisasi teks ke huruf kecil (*lowercasing*).

0. Persiapan: Import Library & Konfigurasi Path

```
In [1]: # Mengimpor seluruh library yang akan digunakan sepanjang notebook
# Library standar Python diimpor terlebih dahulu, diikuti oleh libr

import os
import json
import re
import glob
from pathlib import Path

import pandas as pd
import numpy as np
from tqdm import tqdm

print("Library berhasil diimpor.")
print(f"  pandas   : {pd.__version__}")
print(f"  numpy    : {np.__version__}")
```

```
Library berhasil diimpor.
  pandas   : 3.0.3
  numpy    : 2.4.6
```

```
In [2]: # Mendefinisikan path-path utama yang akan digunakan secara konsist
# di seluruh notebook. Pendekatan ini memudahkan jika suatu saat fo
# project perlu dipindahkan – cukup ubah BASE_DIR di sini.

BASE_DIR = Path("/Users/javierihsan/Javier/S1 UNAIR/skripsi/codinga

# Folder tempat file JSON mentah hasil unduhan dari repo CISA disim
RAW_DATA_DIR = BASE_DIR / "data" / "raw" / "csaf_json"

# Folder tempat hasil pra-pemrosesan akan disimpan
PROCESSED_DATA_DIR = BASE_DIR / "data" / "processed"

# Nama file output hasil pra-pemrosesan
OUTPUT_FILENAME = "data_bersih.csv"
```

```
# Verifikasi bahwa folder-folder tersebut sudah ada
print("Verifikasi path:")
print(f"    RAW_DATA_DIR      : {RAW_DATA_DIR}")
print(f"    PROCESSED_DATA_DIR: {PROCESSED_DATA_DIR}")
print()

if not RAW_DATA_DIR.exists():
    print("PERINGATAN: Folder data/raw/csaf_json tidak ditemukan!")
    print("Pastikan sudah menjalankan setup_project.sh dan menaruh")
else:
    jumlah_json = len(list(RAW_DATA_DIR.glob("**/*.json")))
    print(f"Folder ditemukan. Terdeteksi {jumlah_json} file JSON.")

# Buat folder processed jika belum ada
PROCESSED_DATA_DIR.mkdir(parents=True, exist_ok=True)
print("Folder processed siap.")
```

Verifikasi path:

```
RAW_DATA_DIR      : /Users/javierihsan/Javier/S1 UNAIR/skripsi/cod
ingan_skripsi_javier/data/raw/csaf_json
PROCESSED_DATA_DIR: /Users/javierihsan/Javier/S1 UNAIR/skripsi/cod
ingan_skripsi_javier/data/processed
```

Folder ditemukan. Terdeteksi 3462 file JSON.

Folder processed siap.

1. Pemuatan Data (Data Loading)

Tahap ini membaca seluruh file JSON dari repo CISA Advisory. Setiap file merepresentasikan satu advisory keamanan siber yang berisi metadata, deskripsi kerentanan, rekomendasi mitigasi, dan lain-lain.

```
In [3]: # Mengumpulkan semua path file JSON secara rekursif dari seluruh su
# Struktur folder CISA membagi file berdasarkan tahun (misal: 2023/
# sehingga perlu dilakukan pencarian rekursif dengan glob pattern *

semua_path_json = sorted(RAW_DATA_DIR.glob("**/*.json"))

print(f"Total file JSON ditemukan: {len(semua_path_json)}")

if len(semua_path_json) > 0:
    print("\nContoh 5 file pertama:")
    for p in semua_path_json[:5]:
        print(f"    {p.name}")
```

Total file JSON ditemukan: 3462

Contoh 5 file pertama:

```
icsa-10-147-01.json
icsa-10-316-01a.json
icsa-10-322-01.json
icsa-10-322-02a.json
icsa-10-337-01.json
```

```

In [4]: # Mendefinisikan fungsi untuk mengekstrak field-field teks yang rel
# dari satu file JSON CISA Advisory (format CSAF 2.0).
#
# Field yang diekstrak dipilih berdasarkan relevansinya terhadap an
# - vulnerability_summary : ringkasan inti kerentanan
# - remediation_details : langkah perbaikan yang direkomendasikan
# - notes_general : catatan umum termasuk rekomendasi CISA

def ekstrak_teks_dari_advisory(path_file: Path) -> dict | None:
    """
    Membaca satu file JSON CISA Advisory dan mengekstrak teks yang
    Mengembalikan dictionary atau None jika file tidak valid.
    """
    try:
        with open(path_file, "r", encoding="utf-8") as f:
            data = json.load(f)

        doc = data.get("document", {})
        vuln_list = data.get("vulnerabilities", [])

        # --- Metadata Advisory ---
        advisory_id = doc.get("tracking", {}).get("id", "UNKNOWN")
        advisory_title = doc.get("title", "")
        release_date = doc.get("tracking", {}).get("initial_relea

        # --- Teks Ringkasan Kerentanan ---
        # Mengambil summary dari setiap kerentanan yang terdaftar d
        vulnerability_summaries = []
        cve_ids = []
        cwe_names = []

        for vuln in vuln_list:
            cve_ids.append(vuln.get("cve", ""))
            cwe_names.append(vuln.get("cwe", {}).get("name", ""))

            for note in vuln.get("notes", []):
                if note.get("category") == "summary":
                    vulnerability_summaries.append(note.get("text",

        # --- Teks Detail Remediasi ---
        remediation_texts = []
        for vuln in vuln_list:
            for rem in vuln.get("remediations", []):
                if rem.get("details"):
                    remediation_texts.append(rem["details"])

        # --- Teks Catatan Umum (Rekomendasi CISA) ---
        notes_general = []
        for note in doc.get("notes", []):
            if note.get("category") in ["summary", "general"]:
                if note.get("text"):
                    notes_general.append(note["text"])

        # --- Menggabungkan Semua Teks Relevan menjadi Satu Dokumen
        # Penggabungan ini yang akan menjadi input utama untuk BERT

```

```

gabungan_teks = " ".join(
    vulnerability_summaries + remediation_texts + notes_gen
).strip()

if not gabungan_teks:
    return None

return {
    "advisory_id"           : advisory_id,
    "advisory_title"       : advisory_title,
    "release_date"         : release_date,
    "cve_ids"              : ", ".join(cve_ids),
    "cwe_names"            : ", ".join(set(cwe_names) - {
    "vulnerability_summary" : " ".join(vulnerability_summa
    "remediation_details"   : " ".join(remediation_texts),
    "notes_general"         : " ".join(notes_general),
    "teks_gabungan"        : gabungan_teks,
    "nama_file"             : path_file.name,
}

except json.JSONDecodeError:
    # File JSON rusak atau tidak bisa dibaca, lewati saja
    return None
except Exception as e:
    # Tangkap error lain yang tidak terduga
    print(f" Error pada file {path_file.name}: {e}")
    return None

print("Fungsi ekstraksi berhasil didefinisikan.")

```

Fungsi ekstraksi berhasil didefinisikan.

In [5]: *# Menguji fungsi ekstraksi pada satu file sampel sebelum dijalankan*
ke seluruh dataset. Ini penting untuk memastikan fungsi berjalan
dengan benar sebelum memproses ribuan file.

```

if len(semua_path_json) == 0:
    print("Tidak ada file JSON untuk diuji. Pastikan folder data/ra
else:
    file_sampel = semua_path_json[0]
    hasil_sampel = ekstrak_teks_dari_advisory(file_sampel)

    print(f"Uji ekstraksi pada file: {file_sampel.name}")
    print("-" * 60)

    if hasil_sampel:
        for key, value in hasil_sampel.items():
            # Potong tampilan agar tidak terlalu panjang
            tampil = str(value)[:120] + "..." if len(str(value)) >
            print(f" {key:<25}: {tampil}")
    else:
        print("Hasil ekstraksi kosong – file mungkin tidak memiliki

```

Uji ekstraksi pada file: icsa-10-147-01.json

```
-----
advisory_id          : ICSA-10-147-01
advisory_title       : Cisco Network Building Mediator
release_date        : 2010-02-27T07:00:00.000000Z
cve_ids             : CVE-2010-0595, CVE-2010-0596, CVE-2010-0597, CVE-2010-0598, CVE-2010-0599, CVE-2010-0600
cwe_names           : Use of Default Credentials, Exposure of Sensitive Information to an Unauthorized Actor, Improper Access Control, Missing...
vulnerability_summary : Cisco Mediator Framework 1.5.1 before 1.5.1.build.14-eng, 2.2 before 2.2.1.dev.1, and 3.0 before 3.0.9.release.1 on the ...
remediation_details  : Cisco has provided information on vulnerability workarounds they have also released free software updates that address t...
notes_general       : This CSAF advisory was extracted from unstructured data and may contain inaccuracies. If you notice any errors, please r...
teks_gabungan       : Cisco Mediator Framework 1.5.1 before 1.5.1.build.14-eng, 2.2 before 2.2.1.dev.1, and 3.0 before 3.0.9.release.1 on the ...
nama_file           : icsa-10-147-01.json
```

In [6]: *# Menjalankan proses ekstraksi ke seluruh file JSON yang ditemukan.
Progress bar ditampilkan menggunakan tqdm agar bisa memantau
kemajuan proses, terutama karena jumlah file bisa mencapai ribuan*

```
print(f"Memulai ekstraksi dari {len(semua_path_json)} file JSON...")

records = []
file_dilewati = 0

for path_file in tqdm(semua_path_json, desc="Mengekstrak advisory"):
    hasil = ekstrak_teks_dari_advisory(path_file)
    if hasil is not None:
        records.append(hasil)
    else:
        file_dilewati += 1

# Membuat DataFrame dari semua record yang berhasil diekstrak
df_mentah = pd.DataFrame(records)

print(f"\nEkstraksi selesai.")
print(f" Total file diproses : {len(semua_path_json)}")
print(f" Berhasil diekstrak   : {len(df_mentah)}")
print(f" File dilewati        : {file_dilewati} (kosong/rusak)")
```

Memulai ekstraksi dari 3462 file JSON...

Mengekstrak advisory: 100%|██████████| 3462/3462 [00:01<00:00, 3394.30it/s]

Ekstraksi selesai.

Total file diproses : 3462

Berhasil diekstrak : 3461

File dilewati : 1 (kosong/rusak)

```
In [7]: # Menampilkan gambaran awal dataset yang baru diekstrak:
# dimensi, tipe data, dan beberapa baris pertama untuk inspeksi vis

print(f"Dimensi dataset: {df_mentah.shape[0]} baris x {df_mentah.sh
print()
print("Tipe data per kolom:")
print(df_mentah.dtypes)
print()
print("Pratinjau 3 baris pertama:")
df_mentah.head(3)
```

Dimensi dataset: 3461 baris x 10 kolom

Tipe data per kolom:

advisory_id	str
advisory_title	str
release_date	str
cve_ids	str
cwe_names	str
vulnerability_summary	str
remediation_details	str
notes_general	str
teks_gabungan	str
nama_file	str
dtype:	object

Pratinjau 3 baris pertama:

Out[7]:	advisory_id	advisory_title	release_date	cve_ids	cwe_names	vi
	0	ICSA-10-147-01	Cisco Network Building Mediator	2010-02-27T07:00:00.000000Z	CVE-2010-0595, CVE-2010-0596, CVE-2010-0597, C...	Use of Default Credentials, Exposure of Sensit...
	1	ICSA-10-316-01A	Intellicom NetBiter WebSCADA Vulnerabilities	2010-08-15T06:00:00.000000Z	CVE-2010-4733, CVE-2010-4731, CVE-2010-4732, C...	Improper Control of Generation of Code ('Code ...
	2	ICSA-10-322-01	Ecava IntegraXor Buffer Overflow	2010-08-21T06:00:00.000000Z	CVE-2010-4597	Improper Restriction of Operations within the ...

2. Inspeksi Awal & Penghapusan Duplikat

Sebelum melakukan pembersihan teks, perlu dipastikan tidak ada duplikasi data yang dapat memengaruhi kualitas model topik.

```
In [8]: # Memeriksa jumlah nilai kosong (NaN) pada setiap kolom.  
# Kolom teks_gabungan adalah kolom paling kritis karena inilah  
# yang akan menjadi input BERTopic.  
  
print("Jumlah nilai kosong per kolom:")  
print(df_mentah.isnull().sum())  
print()  
  
# Menghapus baris yang kolom teks_gabungan-nya kosong  
df_mentah = df_mentah.dropna(subset=["teks_gabungan"])  
df_mentah = df_mentah[df_mentah["teks_gabungan"].str.strip() != ""]  
  
print(f"Jumlah baris setelah menghapus teks kosong: {len(df_mentah)}")
```

Jumlah nilai kosong per kolom:

advisory_id	0
advisory_title	0
release_date	0
cve_ids	0
cwe_names	0
vulnerability_summary	0
remediation_details	0
notes_general	0
teks_gabungan	0
nama_file	0
dtype: int64	

Jumlah baris setelah menghapus teks kosong: 3461

```
In [9]: # Memeriksa dan menghapus data duplikat berdasarkan advisory_id.  
# Duplikat bisa muncul jika ada file yang sama tersimpan di beberap  
# subfolder atau ada versi revisi dari advisory yang sama.  
  
jumlah_duplikat = df_mentah.duplicated(subset=["advisory_id"]).sum()  
print(f"Jumlah duplikat berdasarkan advisory_id: {jumlah_duplikat}")  
  
if jumlah_duplikat > 0:  
    # Jika ada duplikat, pertahankan hanya baris pertama (versi awa  
    df_mentah = df_mentah.drop_duplicates(subset=["advisory_id"], k  
    print(f"Duplikat dihapus. Sisa data: {len(df_mentah)} baris")  
else:  
    print("Tidak ditemukan duplikat. Data dapat dilanjutkan.")
```

Jumlah duplikat berdasarkan advisory_id: 0

Tidak ditemukan duplikat. Data dapat dilanjutkan.

```
In [10]: # Menampilkan statistik deskriptif panjang teks pada kolom teks_gab  
# sebelum dilakukan pembersihan. Ini berguna untuk memahami distrib  
# panjang dokumen yang akan diproses oleh BERTopic nantinya.
```

```
df_mentah["panjang_teks_awal"] = df_mentah["teks_gabungan"].str.split()

print("Statistik panjang teks (dalam kata) sebelum pembersihan:")
print(df_mentah["panjang_teks_awal"].describe().round(2))
print()

# Menandai dokumen yang terlalu pendek sebagai kandidat untuk diper
BATAS_MINIMUM_KATA = 5
dokumen_pendek = df_mentah[df_mentah["panjang_teks_awal"] < BATAS_M
print(f"Dokumen dengan kurang dari {BATAS_MINIMUM_KATA} kata: {len(
```

Statistik panjang teks (dalam kata) sebelum pembersihan:

```
count    3461.00
mean      644.29
std      1205.77
min        55.00
25%       351.00
50%       429.00
75%       595.00
max      36777.00
```

Name: panjang_teks_awal, dtype: float64

Dokumen dengan kurang dari 5 kata: 0 dokumen

```
count    3461.00
mean      644.29
std      1205.77
min        55.00
25%       351.00
50%       429.00
75%       595.00
max      36777.00
```

Name: panjang_teks_awal, dtype: float64

Dokumen dengan kurang dari 5 kata: 0 dokumen

3. Pembersihan Karakter Spesial

Tahap ini membersihkan karakter-karakter yang tidak berkontribusi pada makna semantik teks, seperti karakter kontrol, tanda baca berlebih, URL, dan sebagainya.

```
In [11]: # ---- Sub-step 3.1: Menghapus URL ----
#
# URL seperti https://www.cisa.gov/... tidak membawa makna semantik
# yang berguna untuk pemodelan topik dan sebaiknya dihapus.

POLA_URL = re.compile(
    r"http[s]?://(?:[a-zA-Z]|[0-9]|[$-_@.&+]|[*\(\)\,]|(?:%[0-9a-fA
])

def hapus_url(teks: str) -> str:
```



```

return POLA_URL.sub(" ", teks)

# Uji pada teks sampel
contoh_teks = "Visit https://www.cisa.gov/ics for more information about ICS security."
print("Sebelum :", contoh_teks)
print("Sesudah :", hapus_url(contoh_teks))

# Terapkan ke seluruh dataset
df_mentah["teks_bersih"] = df_mentah["teks_gabungan"].apply(hapus_url)
print("\n[OK] URL berhasil dihapus dari seluruh dokumen.")

```

Sebelum : Visit https://www.cisa.gov/ics for more information about ICS security.

Sesudah : Visit for more information about ICS security.

[OK] URL berhasil dihapus dari seluruh dokumen.

```

In [12]: # ---- Sub-step 3.2: Menghapus Tag HTML (jika ada) ----
#
# Beberapa field teks dalam advisory CISA kadang mengandung tag HTML
# seperti <b>, <a href="...">, dll. Tag ini perlu dihilangkan.

POLA_HTML = re.compile(r"<[^>]+>")

def hapus_tag_html(teks: str) -> str:
    return POLA_HTML.sub(" ", teks)

contoh_teks = "The <b>vulnerability</b> allows an attacker to <a href='..>exploit</a> the system."
print("Sebelum :", contoh_teks)
print("Sesudah :", hapus_tag_html(contoh_teks))

df_mentah["teks_bersih"] = df_mentah["teks_gabungan"].apply(hapus_tag_html)
print("\n[OK] Tag HTML berhasil dihapus.")

```

Sebelum : The vulnerability allows an attacker to exploit the system.

Sesudah : The vulnerability allows an attacker to exploit the system.

[OK] Tag HTML berhasil dihapus.

```

In [13]: # ---- Sub-step 3.3: Menghapus Karakter Kontrol & Non-ASCII ----
#
# Karakter kontrol (\n, \t, \r) dan karakter non-ASCII sering kali
# muncul akibat konversi format atau encoding yang tidak konsisten.
# Karakter ini diganti dengan spasi agar tidak memutus kata.

def hapus_karakter_kontrol(teks: str) -> str:
    # Ganti newline, tab, carriage return dengan spasi
    teks = re.sub(r"[\n\t\r]+", " ", teks)
    # Hapus karakter kontrol lain (ASCII 0-31, kecuali spasi)
    teks = re.sub(r"[\x00-\x1F\x7F]", " ", teks)
    return teks

```

```

contoh_teks = "Remote\ncode execution\tvulnerability\r\nin ICS devi
print("Sebelum :", repr(contoh_teks))
print("Sesudah :", repr(hapus_karakter_kontrol(contoh_teks)))

df_mentah["teks_bersih"] = df_mentah["teks_bersih"].apply(hapus_kar
print("\n[OK] Karakter kontrol berhasil dihapus.")

```

Sebelum : 'Remote\ncode execution\tvulnerability\r\nin ICS devices.'

Sesudah : 'Remote code execution vulnerability in ICS devices.'

[OK] Karakter kontrol berhasil dihapus.

```

In [14]: # ---- Sub-step 3.4: Menghapus Karakter Spesial & Tanda Baca Berleb
#
# Karakter seperti @, #, $, %, ^, &, *, (, ), dsb. umumnya tidak
# berkontribusi pada analisis topik berbasis teks. Namun, tanda hub
# dan apostrof dipertahankan karena membentuk istilah teknis sepert
# 'man-in-the-middle' atau "it's".
#
# Catatan: Angka juga dipertahankan karena nomor CVE, CVSS score,
# dan versi firmware merupakan informasi penting dalam konteks keam

def hapus_karakter_spesial(teks: str) -> str:
    # Pertahankan: huruf, angka, spasi, tanda hubung, apostrof, tit
    teks = re.sub(r"^[^a-zA-Z0-9\s\-\'\.\./]", " ", teks)
    # Hapus titik-titik berulang (ellipsis berlebih)
    teks = re.sub(r"\.{2,}", " ", teks)
    # Hapus tanda hubung yang berdiri sendiri (bukan bagian dari ka
    teks = re.sub(r"(?![\w])[-](?![\w])", " ", teks)
    return teks

contoh_teks = "The CVE-2025-1234 vulnerability (CVSS: 9.8) allows r
print("Sebelum :", contoh_teks)
print("Sesudah :", hapus_karakter_spesial(contoh_teks))

df_mentah["teks_bersih"] = df_mentah["teks_bersih"].apply(hapus_kar
print("\n[OK] Karakter spesial berhasil dibersihkan.")

```

Sebelum : The CVE-2025-1234 vulnerability (CVSS: 9.8) allows remote
@ttacker\$ to exploit!!!

Sesudah : The CVE-2025-1234 vulnerability CVSS 9.8 allows remote
ttacker to exploit

[OK] Karakter spesial berhasil dibersihkan.

```

In [15]: # ---- Sub-step 3.5: Normalisasi Spasi Berlebih ----
#
# Setelah berbagai proses penghapusan di atas, kemungkinan besar te
# spasi ganda, tiga, atau lebih yang tersisa. Langkah ini merapikan
# menjadi spasi tunggal dan menghapus spasi di awal/akhir teks.

def normalisasi_spasi(teks: str) -> str:
    # Ganti satu atau lebih spasi/whitespace dengan satu spasi
    teks = re.sub(r"\s+", " ", teks)
    # Hapus spasi di awal dan akhir string
    return teks.strip()

```

```

contoh_teks = " Remote code execution vulnerability "
print("Sebelum :", repr(contoh_teks))
print("Sesudah :", repr(normalisasi_spasi(contoh_teks)))

df_mentah["teks_bersih"] = df_mentah["teks_bersih"].apply(normalisa
print("\n[OK] Spasi berlebih berhasil dinormalisasi.")

```

```

Sebelum : ' Remote code execution vulnerability '
Sesudah : 'Remote code execution vulnerability'

```

[OK] Spasi berlebih berhasil dinormalisasi.

In [16]: *# Verifikasi hasil pembersihan karakter spesial dengan membandingkan
teks sebelum dan sesudah pada beberapa dokumen sampel.*

```

print("=== Verifikasi Pembersihan Karakter Spesial ===")
print()

for i in range(min(3, len(df_mentah))):
    print(f"-- Dokumen {i+1} ({df_mentah.iloc[i]['advisory_id']}) -")
    print(f"  Sebelum: {df_mentah.iloc[i]['teks_gabungan'][:200]}..")
    print(f"  Sesudah: {df_mentah.iloc[i]['teks_bersih'][:200]}..."
    print()

```

=== Verifikasi Pembersihan Karakter Spesial ===

-- Dokumen 1 (ICSA-10-147-01) --

Sebelum: Cisco Mediator Framework 1.5.1 before 1.5.1.build.14-eng, 2.2 before 2.2.1.dev.1, and 3.0 before 3.0.9.release.1 on the Cisco Network Building Mediator NBM-2400 and NBM-4800 and the Richards-Zet a Medi...

Sesudah: Cisco Mediator Framework 1.5.1 before 1.5.1.build.14-eng 2.2 before 2.2.1.dev.1 and 3.0 before 3.0.9.release.1 on the Cisco Network Building Mediator NBM-2400 and NBM-4800 and the Richards-Zeta Mediat...

-- Dokumen 2 (ICSA-10-316-01A) --

Sebelum: WebSCADA WS100 and WS200, Easy Connect EC150, Modbus RTU - TCP Gateway MB100, and Serial Ethernet Server SS100 on the IntelliCom NetBiter NB100 and NB200 platforms have a default username and password...

Sesudah: WebSCADA WS100 and WS200 Easy Connect EC150 Modbus RTU TCP Gateway MB100 and Serial Ethernet Server SS100 on the IntelliCom NetBiter NB100 and NB200 platforms have a default username and password whic...

-- Dokumen 3 (ICSA-10-322-01) --

Sebelum: Stack-based buffer overflow in the save method in the IntegraXor.Project ActiveX control in igcomm.dll in Ecava IntegraXor Human-Machine Interface (HMI) before 3.5.3900.10 allows remote attackers to e...

Sesudah: Stack-based buffer overflow in the save method in the IntegraXor.Project ActiveX control in igcomm.dll in Ecava IntegraXor Human-Machine Interface HMI before 3.5.3900.10 allows remote attackers to exe...

4. Normalisasi Teks: *Lowercasing*

Mengubah seluruh teks menjadi huruf kecil (*lowercase*) agar kata yang sama dengan kapitalisasi berbeda (misal: `Vulnerability` dan `vulnerability`) diperlakukan sebagai token yang identik oleh model.

```
In [17]: # ---- Lowercasing ----
#
# Proses ini mengkonversi seluruh karakter huruf besar menjadi huru
# Langkah ini penting untuk konsistensi representasi teks, terutama
# model embedding tidak membedakan 'Buffer' dengan 'buffer' sebagai
# entitas yang berbeda.
#
# Catatan: Untuk model berbasis BERT seperti yang digunakan dalam p
# beberapa versi model (uncased) sudah melakukan lowercasing secara
# Namun, normalisasi eksplisit di sini memastikan konsistensi pada

def ubah_ke_huruf_kecil(teks: str) -> str:
    return teks.lower()
```

```

contoh_teks = "Buffer Overflow Vulnerability in SCADA System allows
print("Sebelum :", contoh_teks)
print("Sesudah :", ubah_ke_huruf_kecil(contoh_teks))

df_mentah["teks_bersih"] = df_mentah["teks_bersih"].apply(ubah_ke_h
print("\n[OK] Lowercasing selesai diterapkan ke seluruh dokumen.")

```

Sebelum : Buffer Overflow Vulnerability in SCADA System allows Remote Code Execution.

Sesudah : buffer overflow vulnerability in scada system allows remote code execution.

[OK] Lowercasing selesai diterapkan ke seluruh dokumen.

```

In [18]: # Verifikasi hasil lowercasing – pastikan tidak ada huruf kapital
# yang tersisa pada kolom teks_bersih.

ada_huruf_kapital = df_mentah["teks_bersih"].str.contains(r"[A-Z]")

print(f"Dokumen yang masih mengandung huruf kapital: {ada_huruf_kapital}")

if ada_huruf_kapital == 0:
    print("[OK] Semua teks telah dikonversi ke huruf kecil.")
else:
    print("[PERINGATAN] Masih ada huruf kapital – periksa kolom teks_bersih")

# Tampilkan contoh hasil akhir
print()
print("Contoh teks bersih (3 dokumen pertama):")
for i in range(min(3, len(df_mentah))):
    print(f" [{df_mentah.iloc[i]['advisory_id']}] {df_mentah.iloc[i]['teks_bersih']}")

```

Dokumen yang masih mengandung huruf kapital: 0

[OK] Semua teks telah dikonversi ke huruf kecil.

Contoh teks bersih (3 dokumen pertama):

[ICSA-10-147-01] cisco mediator framework 1.5.1 before 1.5.1.built.14-eng 2.2 before 2.2.1.dev.1 and 3.0 before 3.0.9.release.1 on the cisco network building mediator ...

[ICSA-10-316-01A] webscada ws100 and ws200 easy connect ec150 modbus rtu tcp gateway mb100 and serial ethernet server ss100 on the intellicom netbiter nb100 and nb200 p...

[ICSA-10-322-01] stack-based buffer overflow in the save method in the integraxor.project activex control in igcomm.dll in ecava integraxor human-machine interface hmi...

5. Menghapus Dokumen Terlalu Pendek

Dokumen yang sangat pendek setelah proses pembersihan cenderung tidak membawa informasi yang cukup untuk pemodelan topik dan sebaiknya difilter.

```
In [19]: # Menghitung ulang panjang teks setelah seluruh proses pembersihan
# dan memfilter dokumen yang terlalu pendek.
#
# Batas minimum 10 kata dipilih berdasarkan pertimbangan bahwa
# sebuah kalimat yang bermakna dalam domain keamanan siber
# umumnya terdiri atas setidaknya 10 kata.

BATAS_MINIMUM_KATA_FINAL = 10

df_mentah["panjang_teks_bersih"] = df_mentah["teks_bersih"].str.split().str.join(" ")

jumlah_sebelum = len(df_mentah)
df_bersih = df_mentah[df_mentah["panjang_teks_bersih"] >= BATAS_MINIMUM_KATA_FINAL]
jumlah_sesudah = len(df_bersih)

print(f"Batas minimum panjang teks : {BATAS_MINIMUM_KATA_FINAL} kata")
print(f"Dokumen sebelum filter      : {jumlah_sebelum}")
print(f"Dokumen setelah filter       : {jumlah_sesudah}")
print(f"Dokumen dihapus              : {jumlah_sebelum - jumlah_sesudah}")

print()
print("Statistik panjang teks setelah pembersihan:")
print(df_bersih["panjang_teks_bersih"].describe().round(2))
```

```
Batas minimum panjang teks : 10 kata
Dokumen sebelum filter      : 3461
Dokumen setelah filter       : 3461
Dokumen dihapus              : 0
```

```
Statistik panjang teks setelah pembersihan:
count      3461.00
mean       651.56
std        1153.56
min         55.00
25%        354.00
50%        432.00
75%        604.00
max       30073.00
Name: panjang_teks_bersih, dtype: float64
```

```
In [20]: import numpy as np
import pandas as pd
import re

# =====
# UTILITAS: ANALISIS DISTRIBUSI SEBELUM INTERVENSI
# =====

def analisis_distribusi(df: pd.DataFrame, kolom: str = "panjang_teks_bersih"):
    """Tampilkan ringkasan distribusi panjang teks."""
    s = df[kolom]
    q95 = s.quantile(0.95)
    q99 = s.quantile(0.99)
    print(f"Statistik distribusi panjang teks ({kolom}):")
    print(f"  count : {len(s):,}")
```

```

print(f" mean    : {s.mean():.0f} kata")
print(f" median  : {s.median():.0f} kata")
print(f" std     : {s.std():.0f} kata")
print(f" min     : {s.min():.0f} kata")
print(f" Q25     : {s.quantile(0.25):.0f} kata")
print(f" Q50     : {s.median():.0f} kata")
print(f" Q75     : {s.quantile(0.75):.0f} kata")
print(f" Q95     : {q95:.0f} kata")
print(f" Q99     : {q99:.0f} kata")
print(f" max     : {s.max():.0f} kata")
print(f"\n Dokumen > Q95 ({q95:.0f} kata) : "
      f"{(s > q95).sum()} ({(s > q95).mean()*100:.1f}%)")
print(f" Dokumen > Q99 ({q99:.0f} kata) : "
      f"{(s > q99).sum()} ({(s > q99).mean()*100:.1f}%)")

```

6. Ringkasan Akhir & Penyimpanan Data Bersih

Menyimpan hasil pra-pemrosesan ke file CSV sebagai *data bersih* yang siap digunakan pada tahap berikutnya (Skenario A dan Skenario B BERTopic).

```

In [21]: # Membuat kolom final yang akan disimpan.
# Hanya kolom-kolom relevan yang disertakan untuk menjaga ukuran fi
# dan memudahkan proses pemodelan di notebook selanjutnya.

KOLOM_FINAL = [
    "advisory_id",
    "advisory_title",
    "release_date",
    "cve_ids",
    "cwe_names",
    "teks_gabungan",      # teks mentah asli (untuk referensi)
    "teks_bersih",        # teks yang sudah diproses (input BERTopic)
    "panjang_teks_bersih",
    "nama_file",
]

df_final = df_bersih[KOLOM_FINAL].reset_index(drop=True)

print(f"Dataset final: {df_final.shape[0]} dokumen × {df_final.shap
print()
print("Ringkasan kolom:")
for col in KOLOM_FINAL:
    print(f" {col}")

```

Dataset final: 3461 dokumen × 9 kolom

Ringkasan kolom:

```
advisory_id
advisory_title
release_date
cve_ids
cwe_names
teks_gabungan
teks_bersih
panjang_teks_bersih
nama_file
```

In [22]: *# Menampilkan ringkasan statistik akhir sebelum menyimpan.*
Ini juga bisa digunakan sebagai bagian dari laporan pada bab meto

```
print("=====")
print(" RINGKASAN PRA-PEMROSESAN DATA")
print("=====")
print(f" Total dokumen final          : {len(df_final)}")
print(f" Rata-rata panjang teks         : {df_final['panjang_teks_ber']")
print(f" Median panjang teks             : {df_final['panjang_teks_ber']")
print(f" Teks terpendek                  : {df_final['panjang_teks_ber']")
print(f" Teks terpanjang                 : {df_final['panjang_teks_ber']")

if "release_date" in df_final.columns:
    tahun = pd.to_datetime(df_final["release_date"], errors="coerce")
    print(f" Rentang tahun advisory          : {int(tahun.min())} - {i

print("=====")
```

```
=====
RINGKASAN PRA-PEMROSESAN DATA
=====
Total dokumen final          : 3461
Rata-rata panjang teks       : 651.6 kata
Median panjang teks          : 432.0 kata
Teks terpendek               : 55 kata
Teks terpanjang              : 30073 kata
Rentang tahun advisory       : 2010 - 2025
=====
Median panjang teks          : 432.0 kata
Teks terpendek               : 55 kata
Teks terpanjang              : 30073 kata
Rentang tahun advisory       : 2010 - 2025
=====
```

In [23]: *# Menyimpan data bersih ke file CSV di folder data/processed.*
File ini akan menjadi input untuk notebook Skenario A dan Skenari
Encoding UTF-8 dengan BOM (utf-8-sig) dipilih agar kompatibel
dengan berbagai aplikasi termasuk Microsoft Excel.

```
output_path = PROCESSED_DATA_DIR / OUTPUT_FILENAME

df_final.to_csv(output_path, index=False, encoding="utf-8-sig")

print(f"Data bersih berhasil disimpan di:")
```



```

print(f" {output_path}")
print()
print(f"Ukuran file: {output_path.stat().st_size / 1024:.1f} KB")
print()
print("Pra-pemrosesan data selesai.")
print("File ini siap digunakan sebagai input untuk notebook:")
print(" - 02_skenario_A_embedding_umum.ipynb")
print(" - 03_skenario_B_embedding_domain_spesifik.ipynb")

```

Data bersih berhasil disimpan di:
 /Users/javierihsan/Javier/S1 UNAIR/skripsi/codingan_skripsi_javie
 r/data/processed/data_bersih.csv

Ukuran file: 31762.5 KB

Pra-pemrosesan data selesai.

File ini siap digunakan sebagai input untuk notebook:

- 02_skenario_A_embedding_umum.ipynb
- 03_skenario_B_embedding_domain_spesifik.ipynb

```

In [24]: # Verifikasi akhir: membaca kembali file yang baru disimpan
# dan menampilkan beberapa baris untuk memastikan tidak ada
# masalah encoding atau struktur data.

df_verifikasi = pd.read_csv(output_path, encoding="utf-8-sig")

print(f"Verifikasi file tersimpan: {len(df_verifikasi)} baris, {len(df_verifikasi.columns)} kolom")
print(df_verifikasi[["advisory_id", "teks_bersih", "panjang_teks_bersih"]])

```

Verifikasi file tersimpan: 3461 baris, 9 kolom

```

Out[24]:

```

	advisory_id	teks_bersih	panjang_teks_bersih
0	ICSA-10-147-01	cisco mediator framework 1.5.1 before 1.5.1.bu...	3321
1	ICSA-10-316-01A	webscada ws100 and ws200 easy connect ec150 mo...	1084
2	ICSA-10-322-01	stack-based buffer overflow in the save method...	424
3	ICSA-10-322-02A	heap-based buffer overflow in automated soluti...	468
4	ICSA-10-337-01	stack-based buffer overflow in ntwebserver.exe...	455